



Graphic Era
deemed to be **University**
DEHRADUN

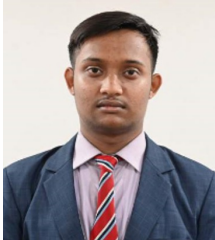
PROJECT AND TEAM INFORMATION

Project Title

(Try to choose a catchy title. Max 20 words).

EcoGraph: Graph-Based Algorithmic Waste Management with Hazard-Aware Routing and Fill-Rate-Adaptive Collection Scheduling

Student/Team Information

<i>Team Name:</i> <i>Team #</i> <i>PBL Mentor:</i>	<i>EcoGraph</i> <i>DSCPP-III-2026-T186</i> <i>Mr. Navneet Rajput</i>
Team member 1 (Team Lead) <i>(Last Name, name: student ID: email, picture):</i>	<i>Jain, Tejas</i> <i>Student ID: 2510011541</i> <i>Roll No. 2028228, Sec A</i> <i>aispace101@gmail.com</i> 
Team member 2 <i>(Last Name, name: student ID: email, picture):</i>	<i>Negi, Apoorv</i> <i>Student ID: 2510011088</i> <i>Roll No. 2027640, Sec D</i> <i>apoorvnegi17@gmail.com</i> 
Team member 3 <i>(Last Name, name: student ID: email, picture):</i>	<i>Nigam, Yash</i> <i>Student ID: 2510010441</i> <i>Roll No. 2028289, Sec H</i> <i>nigamyash07@gmail.com</i> 

PHASE-II PROJECT PROGRESS DESCRIPTION (35 pts)

Project Abstract (2 pts)

(Brief restatement of your project's main goal. Max 300 words).

Garbage trucks in most Indian cities still run the same fixed route every day, whether a street has anything to collect or not. Bins that fill fast overflow, bins that barely fill get visited anyway, and hazardous waste like used syringes or old batteries is often picked up by the same truck, on the same roads, as kitchen scraps.

EcoGraph is our attempt to fix that with plain graph algorithms. We model Dehradun as a weighted graph of depots, homes, road junctions and disposal bins, where every road carries a distance in km and a zone tag: residential, commercial, industrial or arterial. On top of that graph the system does three jobs. It works out which homes and bins are actually due for pickup from how fast each one fills. It builds a shortest-path collection route for each truck using Dijkstra's algorithm. And whenever a waste description reads as medical or chemical, it sends a separate truck that is only allowed on industrial and arterial roads, so that waste never passes through a neighbourhood or a market.

Everything runs inside one C++17 server that also serves a browser front end, where you can watch trucks move across the map and step the city forward day by day to see how the schedule holds up.

Updated Project Approach and Architecture (2 pts)

(Describe your current approach, including system design, communication protocols, libraries used, etc.

Max 300 words).

Design. One C++17 process serves the REST API and the front end on one port. `controllers.cpp` handles HTTP and JSON behind a mutex; domain classes (Graph, Dijkstra, Scheduler, classifier, route builders, Simulation) work on an in-memory adjacency-list city loaded from `city.json`.

Routing. RouteStrategy is an abstract base class; the Priority and FIFO subclasses set the stop order.

Protocol. REST over HTTP with JSON, 13 endpoints, one error format.

Front end. Plain JavaScript with no build step, plus a day-by-day simulation, a limited fleet and a city builder.

Libraries. `cpp-httplib`, `nlohmann/json`, `Cytoscape.js`.

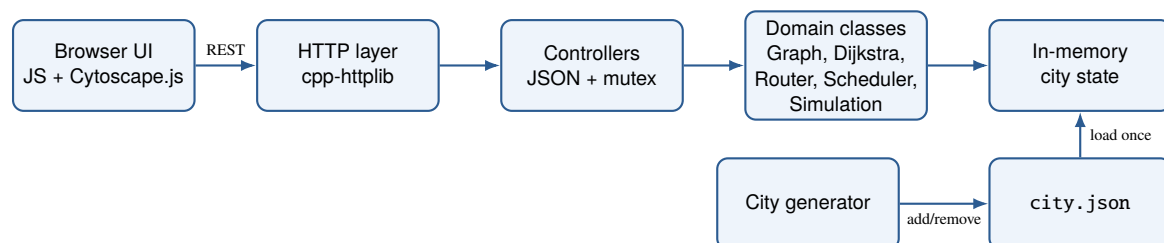
One day, step by step (diagrams (a)–(g) below)

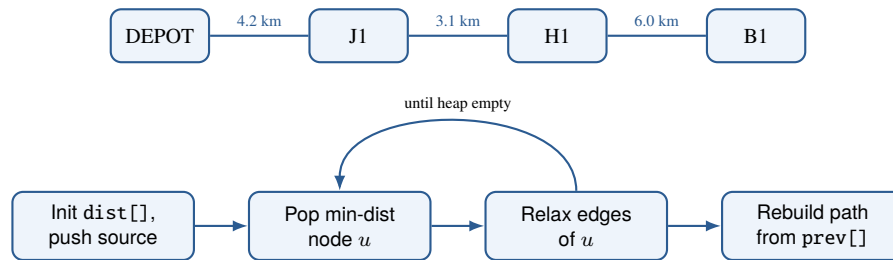
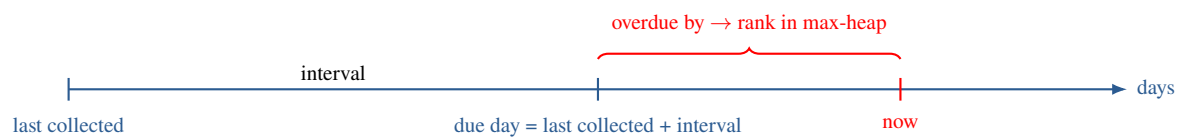
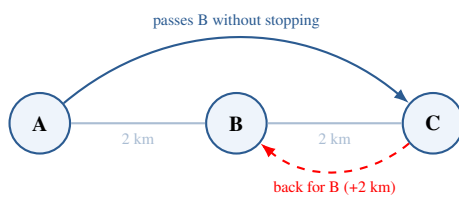
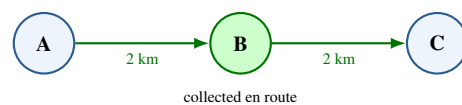
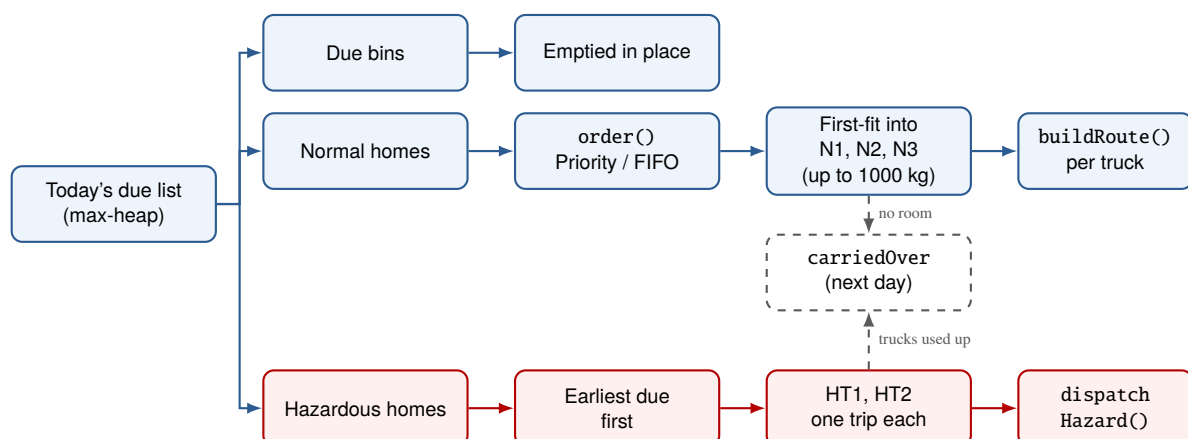
1. The clock moves forward one day.
2. A home is due once the days since its last collection reach capacity / fill rate (1–14 days); the most overdue comes first (c).
3. Due bins are emptied, hazardous homes go to the hazard trucks (f), and normal homes are ordered and packed onto trucks before routing (e).
4. Each truck chains shortest paths in its list order:

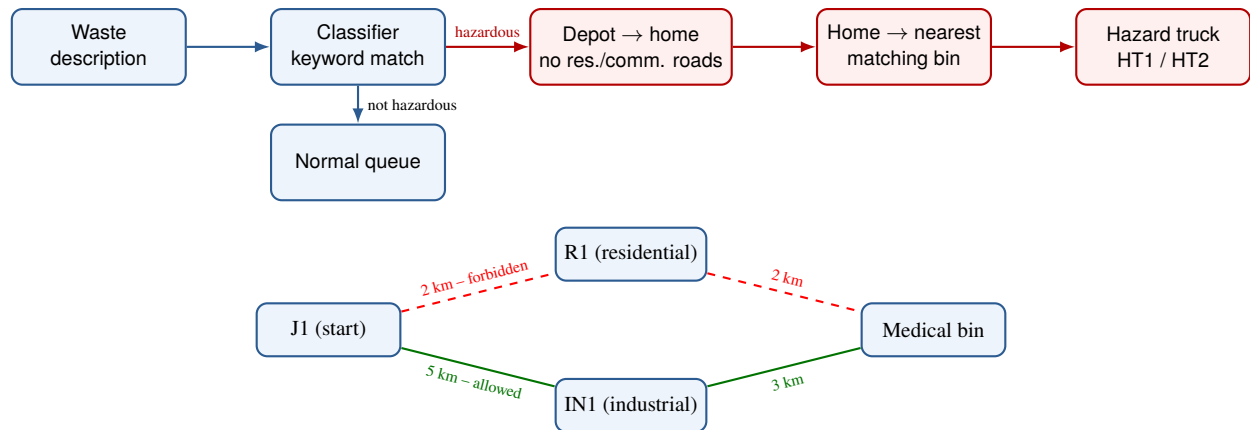
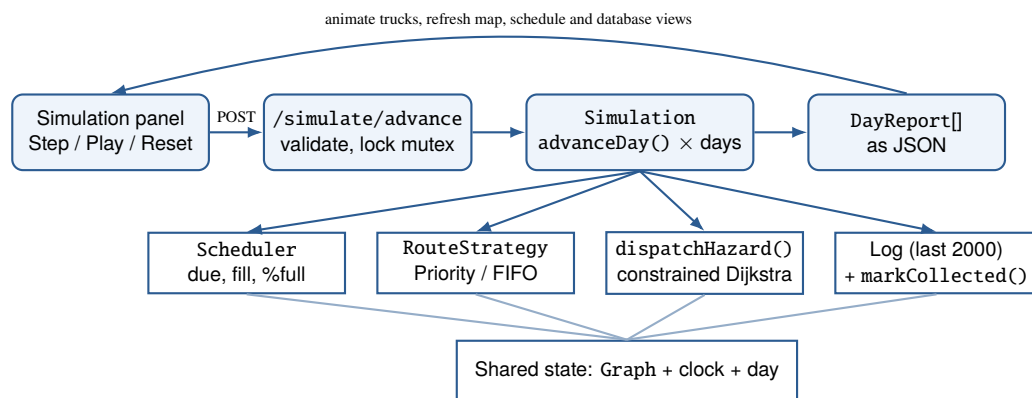
```
current = DEPOT;  visited = { }
for each home in order:
  skip it if in visited
  path = Dijkstra(current → home);  if none, mark unreachable
  empty later stops on the path, add them to visited      (en-route pickup, (d))
  empty home, add to visited;  current = home
then visit the nearest bin for each waste type collected
```

5. Served homes get “last collected = today”; unreachable ones are reported as failed, and carried-over homes rank higher tomorrow.

(a) System Pipeline



(b) Graph Storage and Dijkstra**(c) Fill-Rate Scheduling****(d) En-Route Pickup (new)****Literal order A → C → B: 6 km****EcoGraph: A → B → C: 4 km****(e) Truck Fleet and Capacity (new)**

(f) Automatic Hazard Dispatch and Constrained Dijkstra (new)**(g) Simulation Architecture (new)**

Tasks Completed (7 pts)

(Describe the main tasks that have been assigned and already completed. Max 250 words).

Task Completed	Team Member
Weighted, zone-tagged city graph stored as an adjacency list, and a JSON loader that reads the Dehradun map (51 nodes, 69 roads) at startup	Tejas Jain (Lead)
HTTP server on cpp-httplib that serves the front end and 13 REST endpoints, with one shared error format and a mutex around the shared state	Tejas Jain
Plain Dijkstra and zone-constrained Dijkstra with path rebuilding, used for normal and hazardous routes respectively	Tejas Jain
Priority and FIFO route builders sharing one template method, with en-route pickup and unloading at the nearest matching bin	Tejas Jain
Keyword-based waste classifier and automatic hazard dispatch to the nearest medical or chemical bin, on industrial and arterial roads only	Tejas Jain
Fill-rate scheduler (interval, due, overdue, percent full) with a max-heap that ranks due locations by how overdue they are	Tejas Jain
Day-by-day simulation with a limited fleet, truck capacity, carry-over to the next day and a collection log	Tejas Jain
City generator for adding and removing localities, plus five unit test suites covering every backend module	Tejas Jain
Project README with build steps, folder layout and the full API reference	Apoorv Negi, Yash Nigam
Map view in Cytoscape.js with truck route animation, hazard-route styling and a shape legend	Tejas Jain, Apoorv Negi
Schedule, simulation and database views, the city builder panel and the light/dark theme	Tejas Jain, Yash Nigam

Challenges/Roadblocks (7 pts)

(Describe the challenges that you have faced or are facing so far and how you plan to solve them. Max 300 words).

Road classes beyond residential and commercial. If roads were tagged only as residential or commercial, the hazard truck would have nothing to drive on, because once both are forbidden every road is ruled out. Real cities also have main roads between localities and industrial belts where disposal sites sit, so we added *arterial* and *industrial* road classes. The hazard truck may use these, so it can cross the city without entering a neighbourhood or a market.

Trucks driving past houses on their own list. At first the router followed the scheduler's order literally: given A, C, B with B on the road from A to C, the truck drove past B, reached C, then turned back for B. We added en-route pickup: each leg's path is checked for later stops, which are emptied on the way and skipped when the list reaches them.

Scheduling that never showed its effect. With unlimited trucks, every due home was served every day, so nothing ever overflowed, and choosing Priority or FIFO only changed the stop order, never who got served. We added a limited fleet with 1000 kg per truck and carry-over to the next day. Since an overflowing home heavier than a whole truck would never fit, an empty truck always accepts its first home.

Shared state across requests. cpp-httplib handles requests on several threads, and routing, collection and the simulation all change the same graph, so all shared state sits behind one mutex.

Compiler on Windows. The old MinGW g++ could not build cpp-httplib, which needs proper thread support, so we moved to GCC 13.

Tasks Pending (7 pts)

(Describe the main tasks that you still need to complete. Max 250 words).

Task Pending	Team Member (to complete the task)
Write a multi-stage Dockerfile: the first stage compiles the C++17 server with GCC, the second copies only the binary, <code>city.json</code> and the front-end folder into a slim runtime image	Tejas Jain
Add a <code>.dockerignore</code> that keeps build outputs, report files and the git history out of the image, so it stays small and uploads quickly	Apoorv Negi, Yash Nigam
Choose a host that can run one long-lived container (such as Render, Railway or Fly.io), since the server keeps the city and simulation state in memory	Tejas Jain
Push the image to a container registry and deploy it, with the host's health check pointed at the existing <code>/health</code> endpoint	Tejas Jain
Set the <code>PORT</code> , <code>CITY_FILE</code> and <code>FRONTEND_DIR</code> environment variables on the host. The server already reads these, so no code changes are needed	Apoorv Negi, Yash Nigam
Serve the front end from the same origin as the API, leaving <code>ECO_API_BASE</code> empty in <code>config.js</code> . If it later moves to Vercel, point that value at the API and set <code>CORS_ORIGIN</code>	Tejas Jain
Connect the host to the GitHub main branch so every pushed commit redeploys automatically	Tejas Jain
Add the live demo link and a one-line <code>docker run</code> command to the README	Apoorv Negi, Yash Nigam

Project Outcome/Deliverables (2 pts)

(Describe what are the key outcomes / deliverables of the project. Max 200 words).

By the end of the project we will hand over:

- A C++17 server that loads a city graph and answers routing, hazard dispatch, classification, scheduling and simulation requests over a REST/JSON API.
- A browser front end that draws the Dehradun map, animates each truck's route, highlights hazardous trips, lists what's due today, and lets you run the city forward one day at a time.
- A city builder for adding or removing localities, so the system can be tried on maps other than the one we drew.
- Unit tests for every core module, plus a sample city file someone else can modify.
- A hosted demo link, the README and API docs, and the Phase-II and Phase-III reports and presentations.

The real outcome we care about is simple: given a list of homes, the system should produce a shorter route than a fixed one would, never send hazardous waste through a residential or commercial street, and stop sending trucks to bins that are nowhere near full.

Progress Overview (2 pts)

(Summarize how much of the project is done, what's behind schedule, what's ahead of schedule. Max 200 words.)

We would put the project at roughly 80% done. Everything we promised for Phase-II is working: the graph model, both versions of Dijkstra, the two route builders, the waste classifier, the fill-rate scheduler and the REST API, all with unit tests.

The front end is up as well: the map, route animation, schedule view and database view all work. On top of that we built a day-by-day simulation, a limited truck fleet and a city builder, because a single route on a static map didn't show whether the scheduling actually worked.

Deployment is the one task left. There is no Dockerfile yet, so the demo runs locally for now. Packaging and hosting it is next on our list.

Codebase Information (2 pts)

(Repository link, branch, and information about important commits.)

Repository: github.com/Space-Fighter/EcoGraph **Branch:** main

Layout: server/ (C++ source, headers, tests, data/city.json) and frontend/ (HTML, CSS, JS modules).

Key files

File	What it does
server/src/main.cpp	Starts the server; reads the port, city file and front-end folder
server/src/controllers.cpp	The 13 REST endpoints: parses JSON, calls the domain classes, guards shared state with a mutex
server/src/graph.cpp	City graph: nodes and zone-tagged roads stored as an adjacency list
server/src/dijkstra.cpp	Plain and zone-constrained shortest path
server/src/scheduler.cpp	Collection interval, due, overdue and % full; max-heap of due locations
server/src/waste_classifier.cpp	Keyword rules: medical, chemical, recyclable, compost (else general)
server/src/router.cpp	Priority and FIFO route builders, en-route pickup, nearest-bin unloading, hazard dispatch
server/src/simulation.cpp	Advances days, loads trucks, runs routes and hazard trips, keeps the collection log
server/src/city_loader.cpp	Reads city.json into the graph
server/src/city_generator.cpp	Adds and removes localities for the city builder
server/data/city.json	The Dehradun city: localities, nodes, roads, zones, fill rates
server/tests/	Five unit test programs (Dijkstra, domain, simulation, city data, generator)
frontend/js/graph-view.js	Cytoscape.js map and truck route animation
frontend/js/schedule-view.js, simulation-view.js, database-view.js	Schedule, simulation and database panels
frontend/js/city-builder.js	City builder panel
frontend/js/api.js	Thin fetch wrapper around the REST API

Important commits and the modules they added

Commit	Modules / checkpoint
Graph model and Dijkstra	graph, dijkstra (plain and constrained), Dijkstra tests
REST API server, city data and front end	scheduler, waste_classifier, router (Priority / FIFO), controllers, map view
Day-by-day simulation and database view	simulation, database view, simulation tests
Hazard truck dispatch and limited fleet	automatic hazard trucks, truck count and capacity in simulation and router
City mapped to Dehradun (km/kg units)	real coordinates and localities, city_generator, city builder
Full README with API reference	README, API docs
Locality layout, city builder remove/reset	schematic map layout, remove/reset in the city builder
En-route pickup (latest)	en-route collection in router

Testing and Validation Status (2 pts)

(Provide information about any tests conducted)

Test Type	Status (Pass/Fail)	Notes
Unit: Dijkstra (<code>test_dijkstra</code>)	Pass	Hand-solved graphs; constrained version never uses a forbidden edge; unreachable target handled
Unit: domain (<code>test_domain</code>)	Pass	Classifier rules, scheduler intervals clamped to 1–14 days, Priority vs FIFO ordering and tie-breaks, hazard split
Unit: simulation (<code>test_simulation</code>)	Pass	Truck capacity limits, carry-over to next day, one hazard trip per truck per day
Data: city file (<code>test_city_data</code>)	Pass	Every home reachable, every hazard home has a legal route to its bin
Unit: generator (<code>test_generator</code>)	Pass	Generated localities stay connected; remove and reset behave correctly
Manual API checks (curl and browser)	Pass	<code>/health</code> , <code>/classify</code> , <code>/schedule</code> , <code>/route</code> , <code>/simulate/advance</code> return the expected JSON; bad input gets a clean error

Deliverables Progress (2 pts)

(Summarize the current status of all key project deliverables mentioned earlier. Indicate whether each deliverable is completed, in progress, or pending.)

- C++ backend: routing, hazard dispatch, classifier, scheduler, simulation, REST API – **Completed**
- Browser front end with map, route animation, schedule and database views – **Completed** (polish in progress)
- City builder and Dehradun city data – **Completed**
- Unit tests (5 suites) – **Completed**
- README and API documentation – **Completed**
- Hosted demo (Dockerfile + deployment) – **Pending**
- Phase-II report – **Completed**; Phase-III report, presentation and demo – **Pending**